

Rappel sur les bonnes règles d'écriture du code

NSI TERMINALE - JB DUTHOIT

Coder de façon lisible et claire (spécifications)

Votre code doit être facile à lire par une autre personne. Et cette autre personne peut très bien être-vous deux ans plus tard....on est parfois surpris de ne pas comprendre ce que l'on a bien pu faire - :)!

Pour bien comprendre l'intérêt, il faut garder à l'esprit que vous passerez plus de temps à relire votre code qu'à l'écrire !

Généralités

Votre code doit :

- Beau (c'est mieux que laid :-))
- Explicite
- Simple (c'est mieux que complexe)
- Complexe mieux que compliqué
- Aéré
- Documenté. En Python les commentaires commencent par un `#` .

En python

Vous trouverez tous les détails sur le site officiel de Python.

En résumé :

- En python ne mélangez jamais tabulations et espaces
- Une ligne de code ne devrait pas dépasser 79 caractères.
- Encoder en Utf-8
- Importer les bibliothèques en début de programme sur des lignes séparées.
- Il y a de nombreuses conventions sur les espaces :
 - On met un (et un seul) espace avant et après les affectation, comparaisons, booléens et opérations
 - On ne met pas d'espace pour le reste (, { , [...
- Pour les fonctions, variables et méthodes tout en minuscule avec `_` pour séparer les mots.
- Pour le nom des classes : majuscule pour chaque nouveau mot et mots collés. (Exemple : `MaClasse`)
- En ce qui concerne la documentation à l'intérieur des fonctions :

- Lorsque vous créez une fonction, vous devez la documenter. C'est le rôle du docstring. Le docstring se place juste après la création de la fonction par def. Il commence et termine par trois guillemets "
- Votre docstring doit décrire le rôle de la fonction, puis les paramètres passés en arguments (type et rôle), ainsi que le type de ce qui est retourné
- On accède au docstring en tapant :

```
nom_de_ma_fonction.__doc__
```

1.1

Affectation

NSI TERMINALE - JB DUTHOIT

 Exercice 1.1

L'indice de Masse Corporel est un nombre réel utilisé en médecine. Sa formule est pour une masse en kilos et une taille en mètres :

$$IMC = \frac{masse}{taille^2}$$

1. Écrire en langage Python une fonction `masse_corporelle()` qui :
 - Demande la masse de l'utilisateur en kg (nombre réel)
 - Demande la taille de l'utilisateur en cm (nombre entier).
 - affiche l'IMC de l'utilisateur
2. Écrire en langage Python une fonction `masse_corporelle_2(masse,taille)` qui a comme paramètre `masse` en kg et `taille` en mètres. La fonction renvoie l'indice de masse corporelle.

Testez votre programme avec différentes valeurs.

 Exercice 1.2

Écrire une fonction nommée `dire_bonjour(nom)` ayant comme paramètre une chaîne de caractère `nom` et qui affiche bonjour suivi du mot.

Exemple : `dire_bonjour("Paul")` affiche "Bonjour Paul".

Remarque



Il ne faut surtout pas confondre :

- Une fonction qui **affiche** quelque chose, et dans ce cas on utilisera la commande **print**. Dans ce cas, la fonction est d'ailleurs plutôt appelée **procédure** (car elle ne renvoie rien - en réalité, elle renvoie **None**)
- Une fonction qui **renvoie** quelque chose, et dans ce cas on utilise la commande **return**

 Exercice 1.3

L'énergie cinétique d'un objet de masse m (en kg) qui se déplace à la vitesse v (en m/s) est donnée par la formule :

$$E_c = \frac{1}{2} \times m \times v^2$$

Écrire une fonction nommée `get_energie_cinetique(m,v)` ayant comme paramètres la masse `m` et la vitesse `v` qui renvoie l'énergie cinétique.

Quelle est l'énergie (en Joule) d'un objet de 3 kg se déplaçant à 1.56 m/s ?

 Exercice 1.4

Écrire une fonction `div_euclidienne(a,b)` qui prend en paramètre deux nombres entiers `a` et `b`, qui effectue la division euclidienne de `a` par `b` et qui renvoie le couple `(r,q)`, respectivement le reste et le quotient de cette division euclidienne.

1.2

Instructions conditionnelles

NSI TERMINALE - JB DUTHOIT

● Exercice 1.5



Écrire une fonction python `renverse(a,b)` qui prend en paramètre deux entiers `a` et `b` et qui retourne `b` et `a` (en inversant l'ordre).

Par exemple `renverse(a,b)` va renvoyer `(b,a)`

● Exercice 1.6

On considère la fonction suivante.

```
def examen(x,y,z,t):
    m = (2 * x + 2 * y + z + t) / 6
    if m >= 10:
        print("Le candidat est reçu")
    else :
        print("le candidat est refusé")
    return None
```

1. Documenter la fonction en ajoutant un docstring.
2. Proposer des pré-conditions écrites sur les variables `x`, `y`, `z` et `t` en utilisant l'instruction `assert` qui assurent le bon usage de cette fonction `examen`.

● Exercice 1.7



Écrire une fonction `signe_prod(a,b)` qui reçoit deux entiers `a` et `b` relatifs et qui renvoie `True` si le produit est positif et `False` sinon.

⚠ On ne doit pas calculer le produit des deux nombres.

● Exercice 1.8



Écrire une fonction `categorie(age)` qui a pour paramètre l'âge de l'enfant (qui sera un entier) et qui renvoie sa catégorie :

- "Poussin" de 6 à 7 ans
- "Pupille" de 8 à 9 ans
- "Minime" de 10 à 11 ans
- "Cadet" après 12 ans

● Exercice 1.9



Écrire une fonction Python `est_voyelle(lettre)` pour tester si `lettre` est une voyelle ou non. La fonction renvoie un booléen.

● Exercice 1.10



Écrivez une fonction `triangle(a,b,c)` pour vérifier qu'un triangle est équilatéral, isocèle ou scalène.

Remarque :

Un triangle équilatéral est un triangle dans lequel les trois côtés sont égaux.

Un triangle scalène est un triangle qui a trois côtés inégaux.

Un triangle isocèle est un triangle avec (au moins) deux côtés égaux.

Production attendue :

`triangle(6,8,12)` renvoie "Triangle scalène"

1.3

Les boucles

NSI TERMINALE - JB DUTHOIT

 Exercice 1.11

Écrire une fonction `carre(n)` qui lit en entrée un entier `n` et affiche tous les carrés des nombres inférieurs à `n`, dans l'ordre croissant. Par exemple, si l'entrée est 16, la sortie sera :

```
1
4
9
```

 Exercice 1.12

Créer une fonction `somme(n)` de paramètre `n` et qui retourne la somme des entiers naturels jusque `n`.

- avec une boucle pour
- avec une boucle tant que

 Exercice 1.13

Écrire une fonction `tablemulti(n)` qui reçoit en entrée un nombre entier `n` compris entre 1 et 10 et affiche en sortie la table de multiplication de ce nombre. Par exemple, si la fonction prend comme argument le nombre 7, il affichera la table de multiplication :

```
1 fois 7 = 7
2 fois 7 = 14
3 fois 7 = 21
etc...
```

 Exercice 1.14

Écrire une fonction qui permet d'écrire un tableau d'entiers aléatoires qui s'arrête dès que la somme des nombres dépasse une certaine valeur. la fonction `make_list(deb,fin,maxi)` prend en entrée 3 valeurs entières `deb`, `fin` et `maxi` correspondant respectivement à la plus petite valeur des nombres aléatoires, à la plus grande et à la valeur à ne pas dépasser en faisant la somme de ces nombres.

 Exercice 1.15

Écrire une fonction `moy(notes)` prend en paramètre une liste d'entiers représentant des notes. Le programme doit parcourir cette liste et s'arrêter dès qu'il rencontre un nombre négatif. Il doit ensuite calculer et renvoyer la moyenne des notes lues avant cet arrêt (Attention : le nombre négatif et les nombres qui le suivent éventuellement ne doivent pas compter dans la moyenne!). La fonction donnera un élément de type 'float' en sortie.

 Exercice 1.16

Pour commencer l'ordinateur va choisir au hasard un nombre entier compris entre 1 et 100. L'utilisateur doit alors deviner ce nombre comme ceci : L'utilisateur propose un nombre. L'ordinateur lui dit s'il est trop petit ou trop grand, et ainsi de suite jusqu'à ce que l'utilisateur ait trouvé le bon nombre. Possibilité d'ajouter un compteur qui donnera le nombre d'essais utilisés.

1.4

Les chaînes

NSI TERMINALE - JB DUTHOIT

 Exercice 1.17

 Définissez une fonction `minuchaine(chaine)` qui renvoie le résultat de la conversion de chaîne en minuscules.

 Exercice 1.18

 Définissez une fonction `minuchaine2(chaine)` qui retourne le mot avec la première lettre en majuscule et le reste en minuscule.

 Exercice 1.19

 Écrire une fonction `bien_trie(l)` qui reçoit un tableau `l` de trois chaînes de caractères et qui renvoie `True` si les trois mots sont rangés par ordre alphabétique et `False` Sinon.

 Exercice 1.20

 Écrire une fonction qui prend une chaîne de caractère en entrée et qui retourne la même chaîne mais en ayant au préalable effacé le premier et le dernier caractère. (Vous vous assurerez que la chaîne a une longueur d'au moins 2.) Par exemple, pour l'entrée `Fairy`, une fonction correcte écrira `air`.

 Exercice 1.21

 Écrire une fonction qui prend en entrée une chaîne et qui retourne en sortie la même chaîne mais avec le premier et le dernier caractères échangés. (Vous vous assurerez que la chaîne a comme longueur au moins 2.)

Par exemple, pour l'entrée `Fairy`, une fonction correcte écrira `yairF`.

☛ Indice : utilisez votre solution de l'exercice précédent comme point de départ.

 Exercice 1.22

 Écrire une fonction qui prend en entrée une chaîne de caractères, et qui retourne la chaîne dans l'ordre inverse.

 Exercice 1.23

 Écrire une fonction qui prend en entrée une chaîne de caractères et qui renvoie `True` si la chaîne de caractères est un palindrome, `False` sinon.

☛ Il est possible d'utiliser la fonction précédente !

 Exercice 1.24

Une sous-chaîne est une séquence consécutive de caractères à l'intérieur d'une autre chaîne.

La même sous-chaîne peut se trouver plusieurs fois à l'intérieur de la même chaîne : par exemple :

- 'expertiser' a la sous-chaîne 'er' 2 fois
- 'banane trans-panaméenne' a la sous-chaîne 'an' 4 fois

Écrire une fonction `sous_chaine(aiguille,bottedefoin)` qui prend deux lignes d'entrées, dont la première s'appelle *aiguille* et la seconde *bottedefoin* et qui renvoie nombre de fois que *aiguille* se produit comme une sous-chaîne de *bottedefoin*.

I

1.5

Les tableaux

NSI TERMINALE - JB DUTHOIT

1.5.1 Copie superficielle et copie profonde

 Exercice 1.25

Analyser les lignes suivantes, et prévoir le résultat à l'appel de l1, l2, l3, l4, l5 et l6.

```
# copie superficielle
l1 = [5,6]
l2 = l1
l1.append(7)
l3 = [5,8]
l4 = l3
l4.append(9)
# copie profonde
l5 = [7,8]
l6 = copy.deepcopy(l5)
l5.append(9)
```

1.5.2 Parcours de tableau

 Exercice 1.26

Écrire une fonction `get_max(liste)` qui prend en paramètre un tableau d'entiers et renvoie le max des éléments de celle-ci, sans ordonner la liste.

 Exercice 1.27

Écrire une fonction `get_max_position(l)` qui prend en paramètre un tableau d'entiers `l` et qui renvoie le couple (le tuple) `(pg, ig)` respectivement la plus grande valeur de la liste et la position de cette valeur maximale dans la liste.

 Exercice 1.28

Cette fonction nommée `heure(t,d)` prend deux paramètres :

- un "temps de démarrage" exprimé dans une horloge de 24 heures avec des zéros, comme "08:30" ou "14:07".
- La deuxième ligne est une durée `d` en minutes.

Le programme devra renvoyer la nouvelle heure, c'est-à-dire `d` minutes après l'heure de départ.

Par exemple, pour l'entrée `heure("12:30", 47)`, la sortie correcte serait "13:17". Toutes les heures doivent être formatées sous forme de chaîne de caractères entre "00:00" et "23:59", mais le délai peut aller au-delà de minuit. Par exemple, sur l'entrée `("23:59", 13)`, la sortie correcte est "00:12".

 Exercice 1.29

On s'intéresse à la création de tableaux en Python.

1. Créer un tableau de taille 10 contenant uniquement la valeur `False`. (On pourra utiliser la syntaxe de la multiplication d'une liste.)
2. Créer un tableau de tableaux (tableau 10×10) contenant uniquement la valeur `False`. (On pourra utiliser une boucle ou bien un produit de listes en compréhension.)

3. On définit d'abord

```
l = [False] * 10
tab = []
for i in range(10):
    tab.append(l)
```

Expliquer quel est le problème posé par cette méthode et proposer une correction.

Correction : 1.29

1. On peut utiliser la multiplication de listes en Python :

```
tab = [False] * 10
print(tab)
```

Résultat affiché: [False, False, False, False, False, False, False, False, False, False]

2. Pour un tableau de tableaux (taille 10×10) :
 - Avec une boucle :

```
tab2d = []
for i in range(10):
    tab2d.append([False] * 10)
print(tab2d)
```

- Ou bien directement avec une liste en compréhension :

```
tab2d = [[False] * 10 for i in range(10)]
print(tab2d)
```

3. Ici, la liste `l` est créée une seule fois, puis recopiée 10 fois dans `tab`. En réalité, toutes les lignes de `tab` pointent vers la même liste. Exemple :

```
tab[0][0] = True
print(tab)
```

Résultat : toutes les premières colonnes passent à `True`.

Pour corriger, il faut créer une nouvelle liste à chaque itération :

```
tab = []
for i in range(10):
    l = [False] * 10
    tab.append(l)
```

- Ou bien utiliser une liste en compréhension :

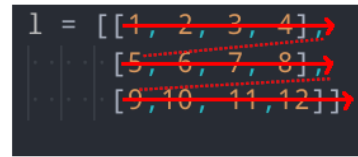
```
tab = [[False] * 10 for i in range(10)]
```

Exercice 1.30

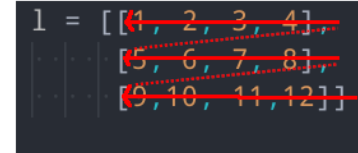
Soit

$$l = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]$$

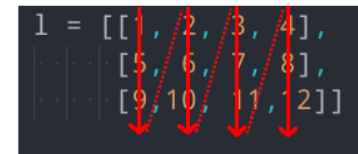
1°) **Parcours Largeur 1** Écrire un algorithme qui parcourt « en largeur » d'abord de gauche à droite puis de haut en bas, chacun des éléments de chacune des sous-listes (de la liste l), en l'affichant dans le Terminal. On pourra utiliser deux boucles **for** imbriquées.



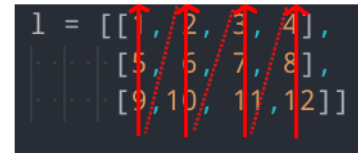
2°) **Parcours Largeur 2** Modifier l'algorithme précédent pour parcourir « en largeur » mais de droite à gauche et de haut en bas, donc tout à l'envers par rapport à la question précédente.



3°) **Parcours Profondeur 1** Parcourir « en profondeur » d'abord de haut en bas et de gauche à droite, chacun des éléments de chacune des sous-listes (de la liste l).



4°) **Parcours Profondeur 2** Parcourir « en profondeur » d'abord de bas en haut et de gauche à droite, donc tout à l'envers par rapport à la question précédente.



1.6

Dictionnaire

NSI TERMINALE - JB DUTHOIT

 Exercice 1.31

Antoine a obtenu les notes suivantes :

```
notes = {'nsi':18,'Maths':16,'SP':13,'Français':14, 'svt':10}
```

1. Créer une fonction `moyenne(d)`, de paramètre un dictionnaire `d`, et qui retourne la moyenne obtenue.
2. Créer une fonction qui produit un affichage du type "relevé de notes" :

```
Antoine a 18 en NSI
Antoine a 16 en Maths
...
Antoine a 10 en SVT
Antoine à ... de moyenne
```

```
notes_bac = {'nsi':18,'maths':16, 'sp':13, 'francais':14}
def moyenne(notes):
    assert type(notes) == dict
    somme = 0
    for i in notes.values():
        somme = somme + i
    return(somme / len(notes))

def releve_notes(notes):
    ''' notes d'Antoine
    Antoine a 18 en nsi
    Antoine a 16 en maths
    ...
    Antoine a 15.25 de moyenne'''
    assert type(notes) == dict
    for i in notes.items():
        print(f"Antoine a {i[1]} en {i[0]}.")
    print(f"Antoine a {moyenne(notes)} de moyenne") #f-string
    print("Antoine a", moyenne(notes), "de moyenne")
```

 Exercice 1.32

Écrire un programme Python pour convertir le nom du mois en un nombre de jours. Accédez à l'éditeur
Sortie attendue :

```
Liste des mois : janvier, février, mars, avril, mai, juin, juillet, août,, Septembre Octobre Novembre
Décembre
Entrez le nom du mois : février
Nombre de jours : 28/29 jours
```

Exercice 1.33



Écrire un programme Python `compteur(d1,d2)` pour combiner deux dictionnaires en ajoutant des valeurs pour les clés communes.

```
d1 = {'a': 100, 'b': 200, 'c': 300}
```

```
d2 = {'a': 300, 'b': 200, 'd': 400}
```

```
assert compteur(d1,d2) == {'a': 400, 'b': 400, 'd': 400, 'c': 300}
```

```
def compteur(d1,d2):
```

```
    d3={}
```

```
    assert type(d1) == dict
```

```
    assert type(d2) == dict
```

```
    for i in d1.items():
```

```
        for j in d2.items():
```

```
            if i[0] == j[0]:
```

```
                d3[i[0]]=i[1]+j[1]
```

```
    for i in d1.items():
```

```
        if i[0] not in d3:
```

```
            d3[i[0]]=i[1]
```

```
    for i in d2.items():
```

```
        if i[0] not in d3:
```

```
            d3[i[0]] = i [1]
```

```
    return d3
```

```
assert compteur({'a':4,'z':5},{'a':7,'e':8}) == {'a':11,'z':5,'e':8}
```

```
assert compteur({'a':100,'b':200,'c':300},{'a':300,'b':200,'d':400})=={'a':400,'b':400,'c':300,'d':400}
```

Exercice 1.34



Écrire une fonction `transformation(chaine)` Python pour créer un dictionnaire à partir d'une chaîne.

Remarque : faire correspondre chaque lettre de la chaîne par son nombre de lettres de la chaîne.

```
transformation("mathématiques") ==
```

```
{'m':2,'a':2,'t':2,'h':1,'é':1,'i':1,'q':1,'u':1,'e':1,'s':1 }
```

```
def transformation(mot):
```

```
    assert type(mot) == str
```

```
    d={} # d = {'m':3}
```

```
    for i in mot:
```

```
        if i not in d:
```

```
            d[i] = 1
```

```
        else:
```

```
            d[i] = d[i] + 1
```

```
    return d
```

```
assert transformation("antoine")== {'a':1,'n':2,'t':1,'o':1,'i':1,'e':1}
```

1.7

Tri

NSI TERMINALE - JB DUTHOIT

 Exercice 1.35

 Écrire une fonction *tri_concatenation*(l1,l2) qui prend en argument deux listes triées A et B, et qui renvoie la liste triée C, C résultant de la concaténation de A et de B.

```
def tri_concatenation(l1,l2):
    '''
    l1 et l2 sont des listes triées
    la fonction retourne une nouvelle liste, qui est la concaténation triée de l1 et l2
    '''
```

 Correction

```
def tri_concatenation(l1,l2):
    '''
    l1 et l2 sont des listes triées
    la fonction retourne une nouvelle liste, qui est la concaténation triée de l1 et l2
    '''
    assert type(l1) == list
    assert type(l2) == list
    for i in range(len(l1)-1):
        assert l1[i] <= l1[i+1]
    for i in range(len(l2)-1):
        assert l2[i] <= l2[i+1]
    l3 = []
    while l1 != [] and l2 != []:
        if l1[0] < l2[0]:
            l3.append(l1[0])
            l1.pop(0)
        else:
            l3.append(l2[0])
            l2.pop(0)
    return l3 + l1 + l2
```

1.8

Autres

NSI TERMINALE - JB DUTHOIT

 Exercice 1.36

Écrire un programme Python pour afficher les nombres qui sont divisibles par 7 et multiple de 5, entre 1500 et 2700 (tous deux inclus)

 Exercice 1.37

Écrire une fonction Python `tableau(m,n)` qui prend en entrée deux entiers m (ligne) et n (colonne) et génère un tableau à deux dimensions. La valeur de l'élément dans la ligne numéro i et la colonne numéro j du tableau doit être égale au produit de i et j .

Remarque : les lignes sont numérotées de 0 à $m - 1$ et les colonnes de 0 à $n - 1$
`tableau(3,4)` renvoie `[[0, 0, 0, 0], [0, 1, 2, 3], [0, 2, 4, 6]]`

 Exercice 1.38

Réécrivez les nombres de l'entrée à la sortie. Arrêtez le traitement de l'entrée après avoir lu le nombre 42. Tous les nombres en entrée sont des entiers à un ou deux chiffres.

Entrée :

2
88
17
42
17
9

Sortie :

2
88
17